Predictive Pricing Analysis for The USA Housing Market

Gordon Oboh

Department of Data Science, Monroe College, King Graduate School

CS703-152W: Applied Data Science Project

Professor Nicholas Nardi

April 18, 2026

Predictive Pricing Analysis for The USA Housing Market

Executive Summary

This project applies the CRISP-DM process to housing listing data covering the USA (no territories are included). The stated goal is twofold: to predict property price and identify important features that affect market value. The target audience is mainly buyers and sellers. The dataset was obtained from Kaggle and was scraped from Craigslist around January 2020. The data understanding and preparation work focused on describing and cleaning the dataset (Regional IQR Price Outliers) and attempting to cluster *lat.&long.* points around cities/metropolitan areas using K-Means. For modeling, the project used Random Forest, Gradient Boosting, and XGBoost. Tree-based models were selected based on EDA.
The overall conclusion recorded in the project pages is that the models performed well on average, but the final recommendation is revision rather than deployment. Suggested next steps include tuning the number of location clusters, collecting more data, exploring GPU-accelerated boosting implementations, and using NLP or transformer methods on the description field.

Figure 1
CRISP-DM pipeline diagram

1 Business Understanding

1.1 Determine Business Objective

The primary objective of this project is to support stakeholders in The USA real estate market by identifying the features and factors that most influence the rental price of a property. The target audience spans the full spectrum of market participants: buyers, sellers, real estate agents (both listing and buyer's agents), brokers, mortgage lenders, homeowners, and renters.

Understanding price drivers empowers selling stakeholders to maximise the value of their listings, while buying stakeholders can use the same insights to identify properties that offer the best return on investment relative to their requirements.

1.2 Assess the Situation

The priorities of each stakeholder group diverge but are anchored to the same question: property value. Sellers consistently seek to maximize the return on their listings, while buyers aim to identify the best value relative to market conditions. Both groups benefit from a clearer understanding of which property attributes drive pricing.

The dataset was sourced from Kaggle and more details can be found in Section 2.1. The key constraints identified during this phase include limited access to additional or more recent data, and the absence of GPU or TPU hardware, which would have supported faster computation and potentially enabled more advanced NLP analysis on listing descriptions.

One of the primary challenges acknowledged at the outset was selecting the appropriate data cleaning, feature engineering, and modeling approaches to reduce noise and surface meaningful, actionable insights from a large and messy real-world dataset.

1.3 Determine Data-Mining Goals

The data mining goals for this project are two-fold:

- Predict rental price using all other available features in the dataset.
- Determine the relative importance of each feature in driving the predicted price.

The modeling strategy used Random Forest, Gradient Boosting, and XGBoost for prediction,

alongside K-Means clustering to engineer location-based features from geographic coordinates. Refer to Section 4.3.1 for more details.

1.4 Produce a Project Plan

This project follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology across six structured phases: Business Understanding, Data Understanding, Data Preparation, modeling, Evaluation, and Deployment.

Key tools and libraries used throughout the project are listed below.

Category	Details
Language / IDE	Python 3.12.3, VS Code, Jupyter Notebook
Key Libraries	pandas 3.0.1, numpy 2.4.2, matplotlib 3.10.8, scipy 1.17.1, scikit-learn 1.8.0, xgboost 3.2.0, category-encoders 2.9.0, joblib 1.5.3
Data Source	Kaggle (Craigslist scrape, Jan 2020)
Hardware	16 GB RAM, Intel Core i5-8350U - Ubuntu 24.04.4 local machine

2 Data Understanding

2.1 Gathering Data

The dataset was obtained from Kaggle. According to the dataset author, the data was scraped from Craigslist in approximately January 2020 and compiled into a single CSV file. The raw file has the following dimensions:

- File size: 558.44 MB
- Total rows: 384,977
- Total columns: 22

2.2 Describing Data

The dataset contains a mix of numeric, categorical, text, and geographic feature types. The target variable for this project is price.

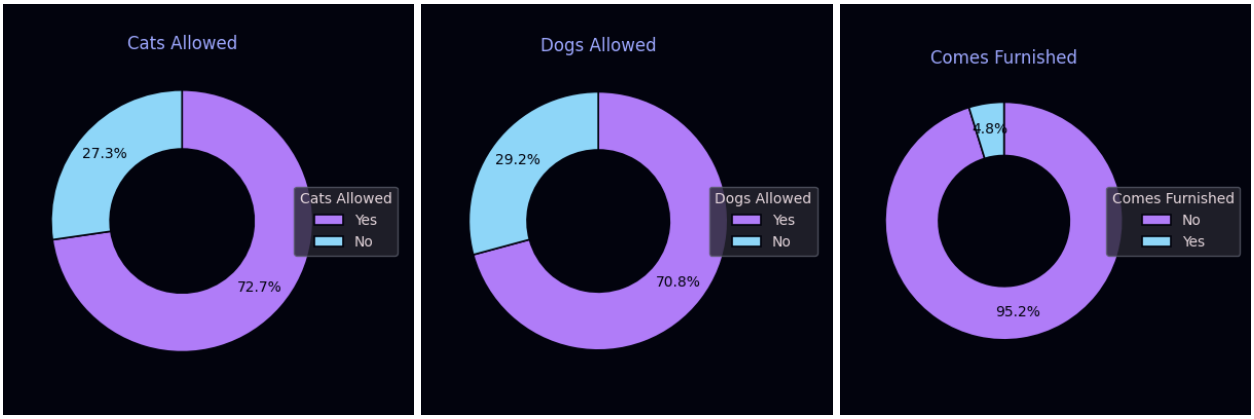
Feature Type	Features	Notes
Numeric	price, sqfeet, beds, baths	Target: price
Categorical (Binary)	cats_allowed, dogs_allowed, smoking_allowed, wheelchair_access, electric_vehicle_charge, comes_furnished	Boolean amenity flags
Categorical (Multi)	region, type, laundry_options, parking_options	Multi-class labels
Geographical	lat, long, state, region	Used for location clustering
Text / URL	id, url, region_url, image_url, description	Excluded from modeling

2.3 Exploring Data

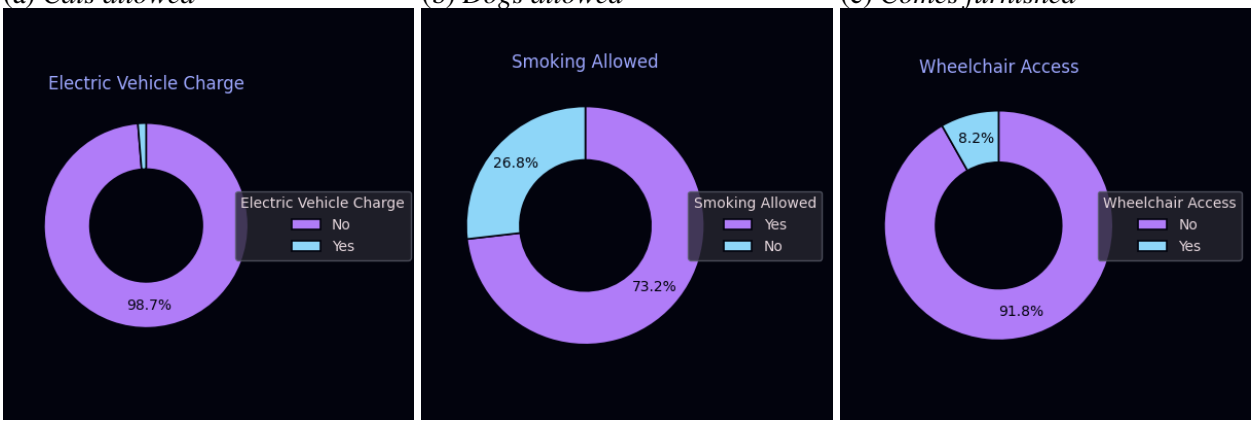
Initial exploratory analysis revealed the following key patterns and distributions within the dataset. Apartments are the most common property type, followed by houses and townhouses. Most properties fall below the third quartile values of 1,150 square feet and $1,395 in price, and most listings are below two bathrooms and two bedrooms at the third quartile level. The dataset is described as largely pet friendly, while furnished listings, EV charging availability, and wheelchair accessibility are all reported as minority cases. About one quarter of listings are noted as smoke friendly.

Figure 2

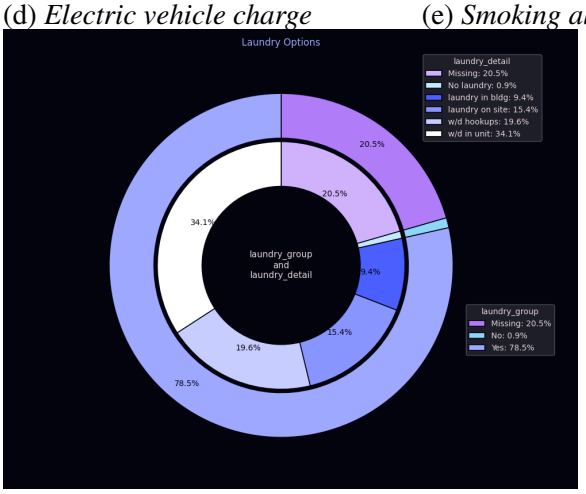
Distribution of selected categorical housing features.



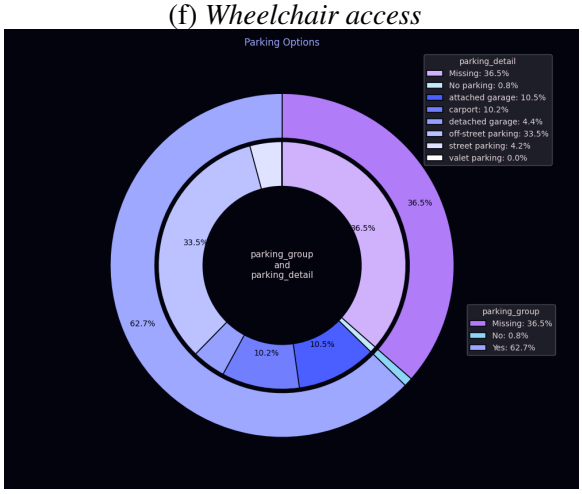
(a) *Cats allowed* (b) *Dogs allowed* (c) *Comes furnished*



(d) *Electric vehicle charge* (e) *Smoking allowed* (f) *Wheelchair access*

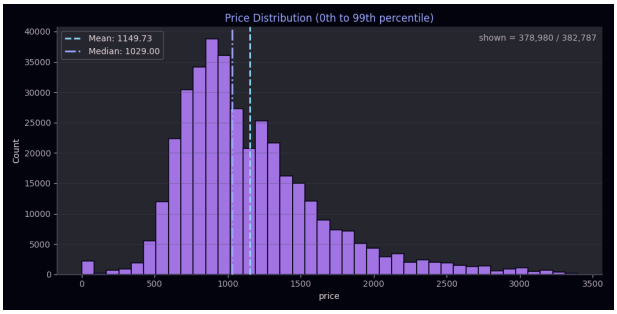


(g) *Laundry options*

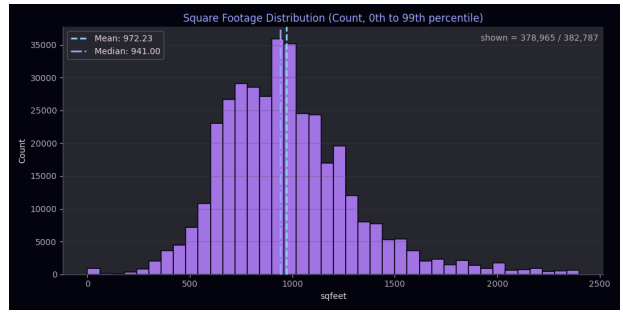


(h) *Parking options*

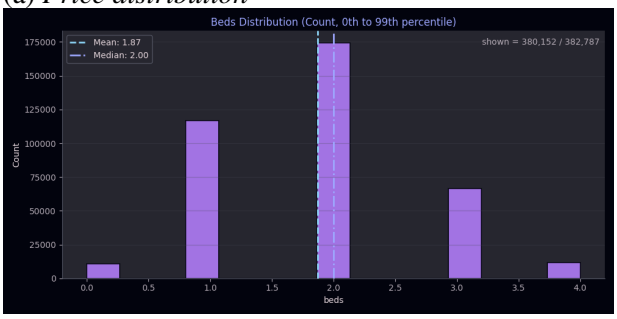
Figure 3
Exploratory data analysis histograms for key housing variables.



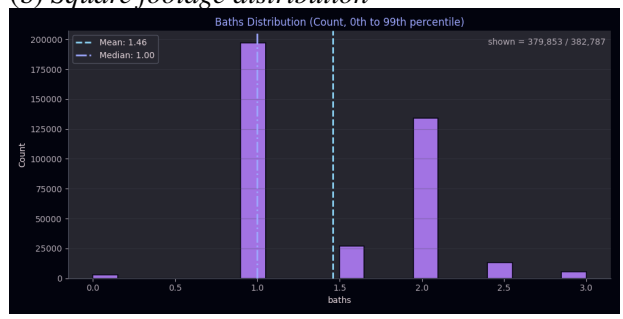
(a) *Price distribution*



(b) *Square footage distribution*

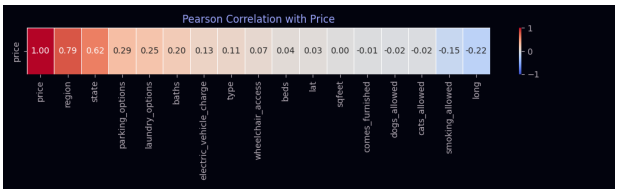


(c) *Beds distribution*



(d) *Baths distribution*

Figure 4
Correlation Test - Numeric Features vs Price



(a) *Pearson*

(b) *Spearman*

2.4 Verifying Data Quality

A structured quality assessment was conducted to identify missing values, duplicates, pricing anomalies, and structural issues before data preparation began.

2.4.1 Missing Values

The following columns were found to contain missing values:

Feature	Missing Count	Missing %
laundry_options	79,026	20.53%
parking_options	140,687	36.54%
lat	1,918	0.50%
long	1,918	0.50%

2.4.2 Anomalies and Outliers

- Properties listed at $0 were identified and removed. These may represent listings open to negotiation, but they introduce noise into price modeling.
- Properties with less than 1 sq ft were identified and removed.
- Properties with 0 bedrooms and 0 bathrooms were retained, as these may represent valid land rental listings.
- Price outliers were noted and addressed in Section 3.2.

3 Data Preparation

3.1 Selecting Data

The selected modeling features are region, price, type, sqfeet, beds, baths, cats_allowed, dogs_allowed, smoking_allowed, wheelchair_access, electric_vehicle_charge, comes_furnished, laundry_options, parking_options, lat, long, and state. The unselected features are id, url, image_url, and description. The selected features were retained because they are more directly relevant to modeling price, while the excluded fields were not carried into the modeling dataset.

3.2 Cleaning Data

The following cleaning steps were applied to prepare the data for modeling: Records with price values below 0 were removed, square footage values below 1 were removed, and latitude/longitude values outside The USA were excluded (Fig 5a & Fig 5b), duplicate rows were checked and that no duplicates were found.

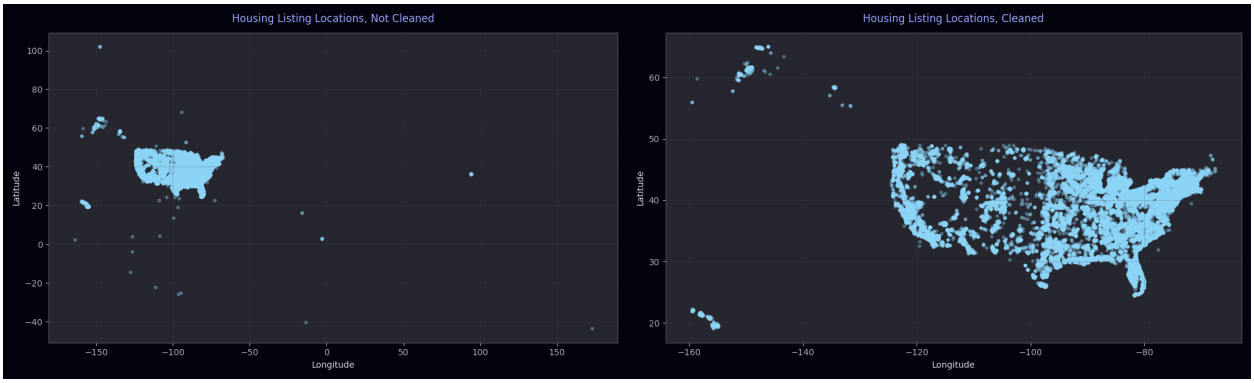
Price outliers were treated using the IQR method because the data is skewed and contains real outliers. Importantly, the outlier rule was applied by state or region rather than across the entire country. The project pages explicitly justify using price alone for outlier removal in order to avoid deleting valid but uncommon listings such as luxury homes, very large homes, properties with five or more bedrooms, or other unusual but legitimate records.

No cleaning was recorded for type, cats_allowed, dogs_allowed, smoking_allowed, wheelchair_access, electric_vehicle_charge, comes_furnished, laundry_options, or parking_options on the basis that these are categorical variables rather than true numeric measurements.

3.3 Constructing Data

A new feature, location_cluster, was engineered from the latitude and longitude columns using K-Means clustering. The intent was to group properties into geographic clusters that approximate proximity to major cities, capturing location-based pricing patterns that raw coordinates or state-level groupings alone cannot express.

Figure 5
Housing Listing Locations



(a) *Not Cleaned* (b) *Cleaned*

K-Means was initially configured with 50 clusters. After evaluating model performance, this was revised to 125 clusters to improve the granularity of location groupings and achieve better predictive results. No separate data-generation step was recorded beyond this feature construction.

3.4 Integrating Data

No data integration was required. The project uses a single, self-contained dataset.

3.5 Formatting Data

No scaling was applied to numeric features, as tree-based models are scale-invariant. Encoding was applied to multi-categorical features: type, laundry_options, parking_options, state, location_cluster, and region. All high-cardinality categorical features were encoded using TargetEncoder from the category-encoders library. This reduced training time from approximately 60–90 minutes per model to under 5 minutes per model, while preserving useful predictive signal. OneHotEncoder would have produced too many features and could distort feature-importance interpretation, while OrdinalEncoder could impose a false order on categories and affect tree split logic Missing values in laundry_options and parking_options were replaced with the category value "missing" before encoding. At the end of this phase, the project pages state that the data was cleaned, structured, and ready for modeling.

4 Modeling

4.1 Selecting Modeling Techniques

Three regression models were selected as the primary predictive models for this project: Random Forest, Gradient Boosting, and XGBoost. The selection was guided by the nature of the dataset: as observed in the correlation heatmap (Fig 4a and Fig 4b), the features do not exhibit a clear linear relationship with price, which makes linear regression models less suitable.

4.1.1 Model Assumptions

- Tree-based models do not assume linearity, normality, or homoscedasticity in the relationship between features and the target.
- These models assume that the target variable is related to the input features in a structured (learnable) way.
- Target encoding introduces an assumption that the historical median/mode of the target variable per category is a meaningful signal.
- K-Means assumes that meaningful geographic clusters exist within the lat./long. space, which is supported by the known concentration of housing markets around urban centers.

4.2 Designing Tests

The test design uses an 80/20 train-test split. The evaluation metrics used are; MAE, MSE, RMSE, R^2, ASE, SSE, and Observed Avg. The visualization outputs used for model assessments are; feature-importance plots, observed-versus-predicted plots, and iteration plots showing ASE versus the number of trees.

A fair comparison between the candidate models is supported by applying the same train-test split and reviewing the same metric set across models.

4.3 Building Model(s)

4.3.1 Model Descriptions

K-Means Clustering: K-Means is an unsupervised algorithm that groups data points into clusters based on similarity. In this project, it was applied to latitude and longitude values to create geographically meaningful clusters. The algorithm iteratively assigns each property to the

nearest centroid and updates centroid positions based on the average coordinates of assigned points until the clusters stabilize. K-Means does not use a target variable; it identifies structure from the input coordinates directly.

Parameter	Definition
n_clusters	Number of cluster groups to create from the coordinates. Initially 50, revised to 125.
random_state=42	Fixes the random seed for centroid initialisation, ensuring reproducible results.
n_init=10	Runs K-Means 10 times with different initialisations; retains the best result to avoid poor clustering from unlucky seeds.

Random Forest Regressor: Random Forest is an ensemble of decision trees trained on random subsets of the training data. For regression, the final prediction is the average of all individual tree predictions. By training each tree on a slightly different sample, the model captures diverse patterns in the data and reduces the variance associated with a single tree. Feature importance is derived from how much each feature reduces prediction error across all trees.

Parameter	Definition
n_estimators=200	Number of decision trees in the forest. More trees improve stability and reduce variance.
random_state=42	Controls randomness in sampling and tree construction for reproducibility.
n_jobs=-1	Uses all available CPU cores to speed up training.

Gradient Boosting Regressor Gradient Boosting builds trees sequentially, where each new tree is trained to correct the residual errors of the previous ensemble. Starting from an initial prediction, the model calculates errors, fits a shallow tree to those errors, and adds it to the model with a small weight controlled by the learning rate. This iterative error-correction process allows the model to capture complex patterns gradually, with shallow trees limiting overfitting at each stage.

Parameter	Definition
n_estimators=200	Number of sequential boosting stages (trees).
learning_rate=0.05	Controls how much each new tree contributes. A lower rate makes learning more gradual and typically improves generalisation.
max_depth=3	Limits the depth of each individual tree, reducing overfitting risk.
random_state=42	Ensures reproducibility.

XGBoost Regressor: XGBoost is an optimised implementation of gradient boosting. It applies the same sequential correction approach but adds regularisation, row/column subsampling, and a highly optimised training engine that makes it faster and often more accurate than standard gradient boosting.

Parameter	Definition
objective="reg:squarederror"	Specifies a regression task minimising squared error.
n_estimators=300	Number of boosted trees.
learning_rate=0.05	Contribution of each tree to the final prediction.
max_depth=6	Maximum tree depth, allowing richer feature interactions than Gradient Boosting.
subsample=0.8	Uses 80% of training rows per tree to reduce overfitting.
colsample_bytree=0.8	Uses 80% of features per tree for better generalisation.
random_state=42	Ensures reproducibility.
n_jobs=-1	Uses all CPU cores for faster training.

Following initial evaluation, two parameter changes were made to improve results:

- K-Means n_clusters: Revised from 50 to 125. Refer to Section 3.3 for more details.
- n_estimators (all regression models): Revised from 200 to 300 to 500. More trees generally improve prediction stability and reduce variance, at the cost of additional training time.

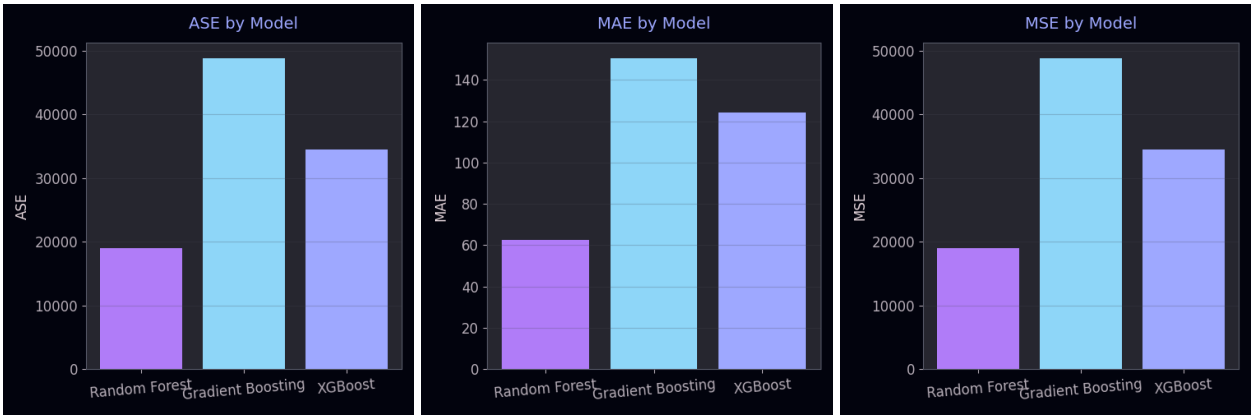
4.4 Assessing Model(s)

Model performance was assessed using the metrics defined in Section 4.2. The table below presents the evaluation results for all three models.

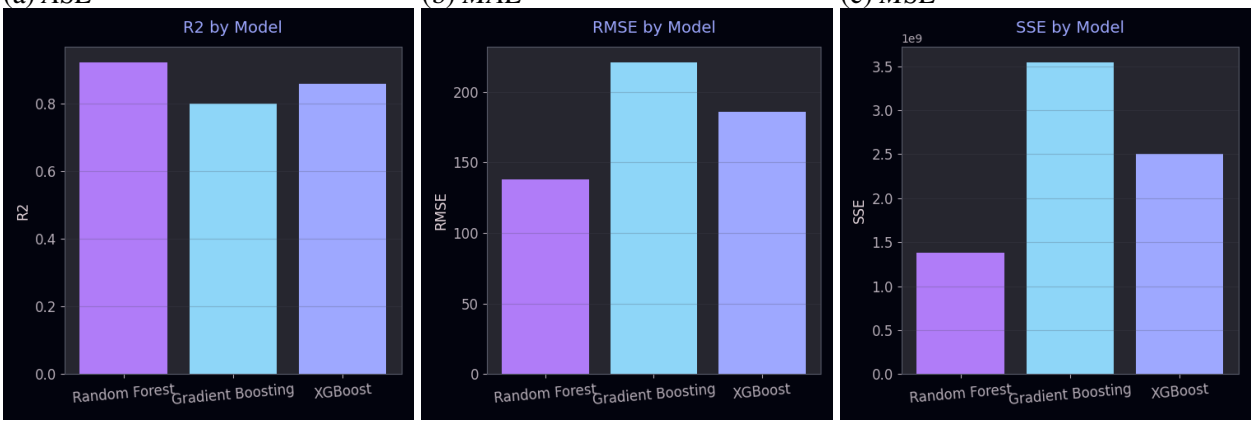
Model	MAE	MSE	RMSE	R^2	ASE	SSE	ObservedAvg
Random Forest	62.7	18989.3	137.8	0.922	18989.3	1377803258.9	1140.94
Gradient Boosting	150.68	48890.3	221.11	0.799	48890.3	3547336339.3	1140.94
XGBoost	124.33	34454.2	185.62	0.858	34454.2	2499892791.2	1140.94

Figure 6

Key Performance Metrics



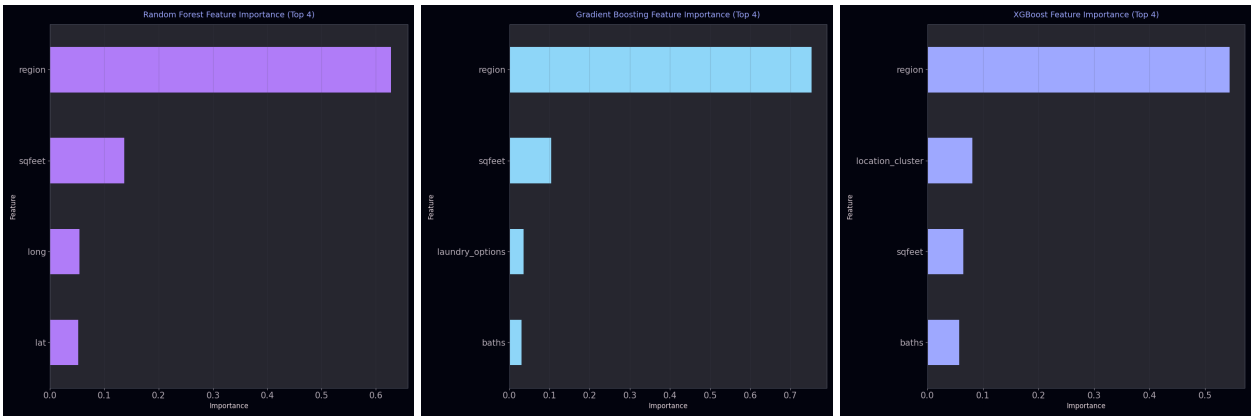
(a) *ASE* (b) *MAE* (c) *MSE*



(d) R^2 (e) *RMSE* (f) *SSE*

Figure 7

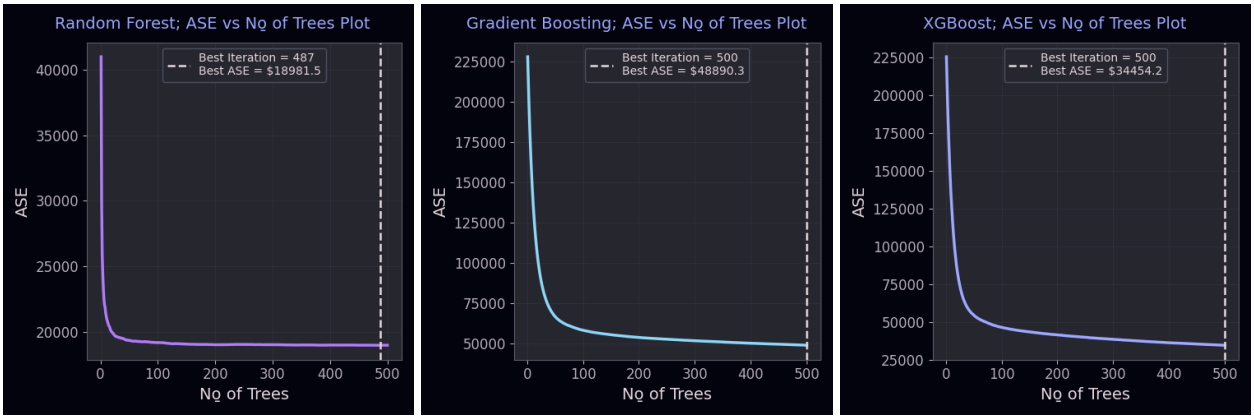
Top 4 Feature Importance Scores Across Tree-Based Models



(a) *Random Forest* (b) *Gradient Boosting* (c) *XGBoost*

Figure 8

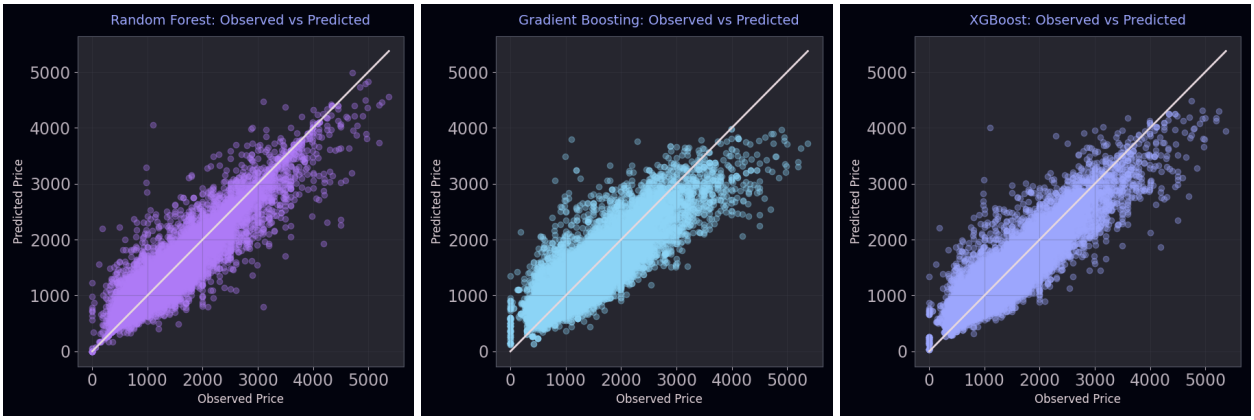
Average Squared Error (ASE) vs Number of Trees for Ensemble Models



(a) *Random Forest* (b) *Gradient Boosting* (c) *XGBoost*

Figure 9

Observed vs Predicted Values



(a) *Random Forest* (b) *Gradient Boosting* (c) *XGBoost*

5 Evaluation

5.1 Evaluating Results

Model comparison shows that the project addresses the prediction problem reasonably well. Random Forest delivered the best overall performance, with the lowest MAE (62.7), MSE/ASE (18989.3), and RMSE (137.8), and the highest R^2, (0.922). XGBoost ranked second, while Gradient Boosting produced the weakest results. This indicates that the cleaned and engineered feature set captured meaningful pricing patterns and that the project was successful in learning useful relationships between housing features and rental price.

For this project, these results mean that Random Forest is the strongest candidate for estimating rental prices from listing data. Its performance suggests that the model can produce predictions that are close to actual prices for many properties in the dataset, making it the most reliable of the three tested approaches. This is important because the main goal of the project is to produce a pricing tool that can help explain market value in a practical housing context.

The feature importance results also show that property size and location-The feature importance results show that region and square footage (sqfeet) were the strongest predictors of rental price across all three models. This is important to the project because it confirms that the models are learning pricing patterns tied to two core drivers of housing value: market location and property size. This suggests that differences in rental price are influenced more by regional market conditions and available living space than by many of the other listing attributes in the dataset. For buyers and renters, this means the model can help assess whether a property is reasonably priced relative to its area and size. For sellers, agents, and other stakeholders, it shows which factors have the greatest influence on expected rental value.

The observed-versus-predicted plots further support the usefulness of the modeling approach, since most predictions follow the general trend of the actual values. However, the wider spread at higher price levels suggests that the models are less precise for more expensive listings. This means the project shows strong promise, but it is not yet ready for deployment without further tuning and validation. Refer to Section 4.4 for more details.

5.2 Reviewing the Process

The process review captures two important lessons. First, the project initially did not use K-Means for location clustering, and the results were described as weaker. Adding location clustering around cities improved the model results.

Second, the project initially applied IQR outlier handling across the entire country. After changing the procedure to apply IQR by region, the project notes that the models improved. This indicates that region-sensitive cleaning better reflected the structure of the housing market data.

5.3 Determining the Next Steps

The following potential improvements were identified for future iterations of this project:

- Optimize K-Means clustering by systematically finding the optimal number of clusters.
- Using newer & larger datasets would improve model relevancy to current market forces.
- Applying NLP to the description field could reveal insights missed by structured features.
- Use GPU/CUDA to speed up training, tuning and hyperparameter search.
- Combining all three models in a weighted ensemble could improve prediction reliability.

On average, all three models performed well, and the modelling approach showed promising predictive performance. However, the project is not yet ready for deployment as-is. The final recommendation is to revise and improve the work before deployment, since the results are encouraging but further tuning, extension, and validation are still needed.

6 Deployment

Deployment activities are explicitly out of scope for this iteration of the project. Phase 6 (which covers deployment planning, monitoring, and maintenance) has not been undertaken at this stage.